Smart Parking

Unique Feature 1: Live Free-Slot Counting from Camera Feed (AI/ML)

Complete Project Documentation - A to Z
every file, every line of code, and how to run it on XAMPP

React.js + TypeScript + Tailwind CSS | Python FastAPI | YOLOv8 + OpenCV | MySQL (XAMPP)

Contents

1. Project overview - what is built
2. How the system works (architecture and data flow)
3. Technologies used and why
4. Folder structure
5. Installation - step by step on XAMPP
6. How to use the website
7. Database design (MySQL)
8. The AI part explained (YOLO + slot comparison)
9. Backend code explained (every file, full code)
10. REST API reference
11. Frontend code explained (every file, full code)
12. Changing the code and rebuilding
13. Testing checklist
14. Troubleshooting
15. What to build next (remaining features)

1. Project overview - what is built

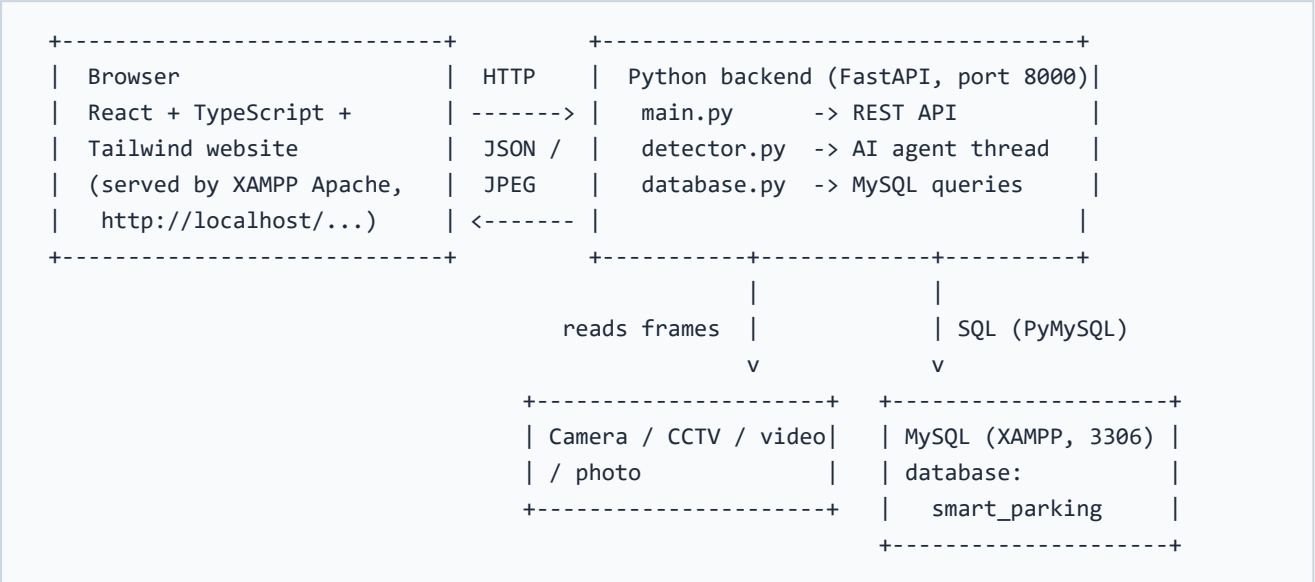
Drivers waste time and fuel circling around looking for an empty parking space. The full Smart Parking idea is a web platform where a driver can see live free slots and reserve one. This first version implements **only Unique Feature 1: Live Free-Slot Counting from Camera Feed**. The other parts (reservations, waitlists, Supabase Auth, overstay anomaly detection with Scikit-learn) are left for later.

What this version does:

- Reads a **single camera feed** - a webcam, a CCTV camera (RTSP URL), a video file or a photo of a parking lot. No sensor is needed under each space.
- A **pre-trained YOLOv8 model** finds every vehicle (car, motorcycle, bus, truck) in the picture.
- The vehicle boxes are compared with **predefined slot coordinates** (rectangles the operator draws once on the website).
- Each slot is marked **free** or **occupied** in the MySQL database automatically, about once per second. When a car leaves, the slot becomes free on the website within a second or two.
- Vehicles that are not inside any slot (for example parked in the driveway) are counted as a simple **anomaly**.
- A history row is saved every 10 seconds so you can see how the lot changes over time.
- The website (served by XAMPP Apache) shows the live AI picture, counters, a colour slot map, history, and a page to draw the slots.

2. How the system works

2.1 Architecture



XAMPP gives us two of the three servers: **Apache** (serves the built website files) and **MySQL** (stores slots and history). XAMPP cannot run Python, so the FastAPI backend runs next to it with **Uvicorn** on port 8000. The website talks to the backend using normal HTTP requests (`fetch`).

2.2 What happens every second (the AI loop)

```
1. detector.py reads ONE frame from the camera source (OpenCV)
2. YOLOv8 looks at the frame  ->  list of vehicle boxes [x1,y1,x2,y2,confidence]
3. Read all slots from MySQL  ->  list of slot rectangles
4. For every slot: how much of the slot is covered by the best-matching vehicle box?
      covered >= 30 %  -> "occupied"      covered < 30 % -> "free"
5. If a slot's status changed  ->  UPDATE slots SET status=... in MySQL
6. Count vehicles that are in no slot  ->  "outside slots" (anomaly)
7. Draw green/red slot boxes + orange vehicle boxes on the picture, keep it in memory
8. Every 10 s  ->  INSERT a row into detection_logs
9. Sleep 1 second, repeat
```

2.3 What happens in the browser

- Every **2 seconds** the dashboard calls `/api/status`, `/api/slots` and `/api/logs` and redraws the numbers, slot map and history.
- Every **1 second** the live picture is reloaded from `/api/frame`.
- On the Slot Setup page the operator drags rectangles on a snapshot; each one is sent to `POST /api/slots` and stored in MySQL.

3. Technologies used and why

Part	Technology	Why / role in this project
Frontend	React.js 19	Builds the user interface from small reusable components (StatCard, SlotMap, ...).
	TypeScript	JavaScript with types. `types.ts` describes the exact JSON the backend sends, so mistakes are caught when building.
	Tailwind CSS 4	Styling by writing small classes such as `bg-white rounded-xl shadow` directly in the JSX - no separate CSS files to maintain.
	Vite	Build tool. Converts the TypeScript/React/Tailwind source into plain HTML/JS/CSS that Apache can serve.
Backend	Python + FastAPI	Web API framework. Python was chosen because YOLO and OpenCV are Python libraries, so the AI runs in the same program as the API.
	Uvicorn	The server program that runs FastAPI on port 8000.
AI / CV	YOLOv8 (Ultralytics)	Pre-trained object detector. `yolov8n-obb.pt` (aerial DOTA dataset) for top-down cameras, or `yolov8n.pt` (COCO dataset) for angled CCTV cameras. "n" = nano, the smallest and fastest version.
	OpenCV	Opens the webcam / video / CCTV stream, reads frames, draws boxes, encodes JPEG pictures.
Database	MySQL (XAMPP)	Stores slots and detection history. The proposal mentions PostgreSQL/Supabase; for local XAMPP use we use MySQL as requested. The SQL is standard, so moving to PostgreSQL later is easy.
	PyMySQL	Pure-Python library that lets the backend talk to MySQL.
Web server	XAMPP Apache	Serves the built website at `http://localhost/smart-parking/` .

Not used yet (planned for later features): Supabase Auth, Scikit-learn, GitHub Actions, Vercel/Render deployment.

4. Folder structure

```
C:\xampp\htdocs\smart-parking\
|-- index.html               -> redirects http://localhost/smart-parking/ to app/
|-- app\                     -> BUILT website (created by "npm run build"; do not edit)
|   |-- index.html
|   +-- assets\  (index-xxxx.js, index-xxxx.css)
|-- database\
|   |-- smart_parking.sql    -> creates the database + 2 tables (import in phpMyAdmin)
|   +-- .htaccess            -> blocks web access to this folder
|-- backend\                 -> Python FastAPI + AI
|   |-- requirements.txt     -> Python libraries to install
|   |-- config.py            -> all settings (DB login, camera source, thresholds)
|   |-- database.py          -> all MySQL queries
|   |-- detector.py          -> the YOLO AI agent (background thread)
|   |-- auto_slots.py        -> finds parking spaces automatically
|   |-- main.py              -> the REST API (FastAPI app)
|   |-- uploads\             -> uploaded photos/videos (created automatically)
|   +-- .htaccess            -> blocks web access to the Python files
+-- frontend\                -> React + TypeScript + Tailwind SOURCE code
    |-- package.json, vite.config.ts, tsconfig.json, index.html
    +-- src\
        |-- main.tsx, App.tsx, api.ts, types.ts, index.css, vite-env.d.ts
        +-- components\
            |-- Dashboard.tsx, StatCard.tsx, LiveFeed.tsx, SlotMap.tsx
            +-- SourcePanel.tsx, LogsTable.tsx, SlotSetup.tsx
```

5. Installation - step by step on XAMPP

Step 1 - Install the programs (only once)

Program	Where	Note
XAMPP	apachefriends.org	Already installed at `C:\xampp`.
Python 3.12	python.org/downloads	During setup tick **"Add python.exe to PATH"**. (Python is **not** installed on this PC yet.)
Node.js	nodejs.org	Only needed if you change the frontend code. Already installed (v24).

Step 2 - Start Apache and MySQL

Open the **XAMPP Control Panel** and press **Start** next to **Apache** and **MySQL**.

Step 3 - Create the database

Open `http://localhost/phpmyadmin` → **Import** tab → choose `C:\xampp\htdocs\smart-parking\database\smart_parking.sql` → **Import**. You will see the database `smart_parking` with tables `slots` and `detection_logs`.

> This step is optional: the backend also creates the database and tables automatically when it starts (see `init_database()`).

Step 4 - Install the Python libraries (once)

Open **Command Prompt** and run:

```
cd C:\xampp\htdocs\smart-parking\backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

`venv` is a private folder for this project's libraries. `ultralytics` also installs PyTorch, so the download is large (about 1-2 GB) and takes a few minutes.

Step 5 - Start the backend

```
cd C:\xampp\htdocs\smart-parking\backend
venv\Scripts\activate
uvicorn main:app --port 8000
```

The first start downloads the YOLO model `yolov8n-obb.pt` (about 6 MB). If no webcam is found you will see "Camera index out of range" - that is fine, upload a photo or video on the website instead. When you see `[AI] model ready` the AI is working. Keep this window open. Test it in the browser: `http://localhost:8000/docs` shows every API endpoint.

Step 6 - Open the website

Go to **`http://localhost/smart-parking/`** (it redirects to `/smart-parking/app/`).

> If XAMPP Apache uses another port (e.g. 8080) open `http://localhost:8080/smart-parking/`. The backend must always be on port 8000, because `frontend/src/api.ts` uses `http://localhost:8000`.

6. How to use the website

A. Choose what the camera is

On **Live Dashboard** → **Camera Source**:

- Type `0` and press **Use** for the laptop webcam (default).
- Type a CCTV address like `rtsp://admin:pass@192.168.1.20:554/stream1` for a real IP camera.
- Or **upload** a parking-lot photo or video. A top-view or high-angle picture of a car park works best (search "parking lot top view" photos/videos, or record one from a balcony). A video loops forever, so cars "arrive" and "leave".

After an upload the AI automatically finds the parking spaces (section 8.4) and the message shows how many it found. Check them on Slot Setup and fix them by hand if needed (step B).

B. Check / draw the slots (once per camera)

Press the green **Auto-detect parking spaces** button to let the AI search again, or draw slots yourself:

Open **Slot Setup**. A snapshot of the camera appears (the whole picture fits on screen). The code box shows the next free name (e.g. `A1`). Choose a shape:

- **Rectangle (drag)** - for straight spaces: press on one corner of the **whole** space, drag to the opposite corner, release.
- **4 corners (click)** - for diagonal / tilted spaces: click the 4 corners of the space on its painted lines. After the 4th click the slot is saved. The click order does not matter.

The slot is saved in MySQL and the code becomes `A2`. Repeat for every space. Wrong slot? Press **Delete**. A slot must cover the full area where a car stands, not the gap between cars.

C. Watch it live

Back on **Live Dashboard**: counters (total, free, occupied, vehicles seen, outside slots), the live AI picture (green = free, red = occupied, orange = vehicle), the slot map, and the history table update by themselves.

7. Database design (MySQL)

Table `slots`

Column	Type	Meaning
id	INT, PK, auto increment	Unique number of the slot
slot_code	VARCHAR(20), UNIQUE	Name shown to users, e.g. A1
x1, y1	INT	Top-left corner of the slot in the camera picture (pixels)
x2, y2	INT	Bottom-right corner (pixels)
points	TEXT, NULL	Only for tilted slots: the 4 corners as JSON text, e.g. `[[10,20],[90,5],[110,200],[30,215]]`. Then x1..y2 is the box around them. NULL = plain rectangle.
status	ENUM('free','occupied')	Set automatically by the AI
updated_at	TIMESTAMP	Updated automatically by MySQL whenever the row changes = time of last status change

Table `detection_logs`

Column	Meaning
id	Row number
total_slots / free_slots / occupied_slots	Lot state at that moment
cars_detected	How many vehicles YOLO saw in the picture
cars_outside_slots	Vehicles that are not inside any slot (anomaly signal)
created_at	When the row was saved

File: database/smart_parking.sql

```
-- ============================================================
--  Smart Parking
--  Feature 1: Live Free-Slot Counting from Camera Feed
--  Database: MySQL (XAMPP)
--  Import this file in phpMyAdmin  (http://localhost/phpmyadmin)
-- ============================================================

CREATE DATABASE IF NOT EXISTS smart_parking
  CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;

USE smart_parking;

-- ------------------------------------------------------------
-- Table: slots
-- One row per parking space. x1,y1 = top-left corner and
-- x2,y2 = bottom-right corner of the slot in the camera image
-- (pixel coordinates). For a tilted slot, "points" holds its
-- 4 corners as text, e.g. [[10,20],[90,5],[110,200],[30,215]],
-- and x1..y2 is the box around them. The AI updates "status".
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS slots (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  slot_code   VARCHAR(20)  NOT NULL UNIQUE,
  x1          INT          NOT NULL,
  y1          INT          NOT NULL,
  x2          INT          NOT NULL,
  y2          INT          NOT NULL,
  points      TEXT         NULL,
  status      ENUM('free','occupied') NOT NULL DEFAULT 'free',
  updated_at  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- ------------------------------------------------------------
-- Table: detection_logs
-- A snapshot of the lot saved every few seconds by the AI.
-- cars_outside_slots = vehicles that are not inside any slot
-- (e.g. parked in the driveway) -> a simple anomaly signal.
-- ------------------------------------------------------------
CREATE TABLE IF NOT EXISTS detection_logs (
  id                  INT AUTO_INCREMENT PRIMARY KEY,
  total_slots         INT NOT NULL,
  free_slots          INT NOT NULL,
  occupied_slots      INT NOT NULL,
  cars_detected       INT NOT NULL,
  cars_outside_slots  INT NOT NULL,
```

```
  created_at          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

8. The AI part explained

8.1 YOLO in simple words

YOLO ("You Only Look Once") is a neural network that looks at a whole picture once and outputs a list of objects, each with a **class** (what it is), a **box** (x1, y1, x2, y2) and a **confidence** (0 to 1). We use **pre-trained** models, so we do **not** need to train anything. Which model is best depends on the camera angle:

Camera	Model (config.py YOLO_MODEL)	Trained on	Vehicle classes	IMAGE_SIZE
Straight down (drone / aerial photo)	`yolov8n-obb.pt` (default)	DOTA aerial dataset	small vehicle, large vehicle	1024
At an angle (normal CCTV)	`yolov8n.pt`	COCO dataset	car, motorcycle, bus, truck	640

Why two models? Tested on a 2250×4000 drone photo with about 150 cars, the normal COCO model found only 5-14 cars (it has rarely seen cars from directly above), while the aerial model found 106. The aerial model returns **rotated boxes** ("OBB" = oriented bounding box); `.xyxy` converts them to normal upright rectangles so the rest of the code is the same for both models.

The code does not hard-code class numbers. It reads the model's class names and keeps those listed in `VEHICLE_NAMES`, so switching model only needs a change in config.py. Detections with confidence below `0.25` are ignored (`CONFIDENCE`).

Automatic image size

YOLO shrinks every picture to a fixed size before looking at it. The aerial model was trained on cars that look **small**. In a photo from a high drone, cars are already small, so a large size (1024) works best. In a photo from a low drone, cars are big, so a small size (320) makes them look the way the model expects. Tests: photo 1 (high drone) found 106 cars at 1024 but few at 320; photo 2 (low drone) found 35 at 320 and 0 at 1024. Therefore, for every new camera source, `choose_image_size()` tries every value in `IMAGE_SIZES` once, prints how many vehicles each found, and keeps the best one.

8.2 Deciding if a slot is occupied - the overlap ratio

Both slots and vehicles are handled as **4-corner shapes** (polygons): a rectangle slot becomes its 4 corners, a tilted slot uses its clicked corners, and the aerial model gives each car as a rotated 4-corner box. For each slot and each vehicle we compute how much of the **slot area** is covered by the vehicle:

```
intersection = area where the slot shape and the vehicle shape overlap
               (OpenCV: cv2.intersectConvexConvex)
overlap      = intersection area / slot area (cv2.contourArea)   (0.0 ... 1.0)
```

```
slot is OCCUPIED  if  the biggest overlap of any vehicle  >= 0.30
otherwise         FREE
```

Worked example. Slot A1 = (100,100)-(200,300) → area = 100 × 200 = 20,000 px. A car box = (120,150)-(230,320). Intersection = (120,150)-(200,300) = 80 × 150 = 12,000 px. Overlap = 12,000 / 20,000 = **0.60** ≥ 0.30 → A1 is **occupied**.

Why 30% and not "the centre of the car is in the slot"? Cameras are usually at an angle, so a car's box spills into the neighbouring slot a little; 30% coverage ignores small spill-over but still catches a parked car. You can tune `OCCUPIED_OVERLAP` in config.py.

8.2b Second opinion: the visual check

Pre-trained YOLO sometimes misses a car (e.g. a shiny black car photographed from very close). So a slot is also marked **occupied** when it **looks busy**:

```
contrast  = standard deviation of brightness inside the inner 70 % of the slot,
            after blurring away fine texture (bricks, asphalt grain)
reference = the LOWEST contrast of all slots in the picture (that slot is almost surely empty)
occupied  = YOLO sees a car in the slot
            OR contrast > max(reference x 1.4, reference + 8)
```

Why it works: bare ground is even; a car has a dark body, windows and shadows. Measured values: close-up photo - empty slots 24-25, car slot 41; busy-lot photo - empty space 8, spaces with cars 19-81. Only the inner 70 % is used, so painted lines and kerbs at the slot edges don't count. It needs at least 2 slots and can be switched off with `VISUAL_CHECK = False`. Limit: a strong tree shadow or an oil stain in an empty space can make it look busy.

8.3 Anomaly - vehicle outside slots

A vehicle whose overlap with **every** slot is below 30% is not parked in a slot (driving through, waiting in the aisle, or parked illegally). The number is shown as "Outside Slots" and saved in the history. This keeps the digital status matched with the physical reality, as the proposal requires.

8.4 Automatic parking-space detection

When a photo/video is uploaded (or **Auto-detect parking spaces** is pressed on Slot Setup), the AI creates the slots itself, named P1, P2, ... (top-to-bottom, left-to-right). Two methods are combined:

Method	How it works	Finds
B. Parked cars (YOLO) - runs first	Every detected car stands in a space, so a space is made around it (same angle as the car, width = "pitch" = usual distance between neighbouring cars). It also produces **hints** for method A: the directions the cars point in (stall lines run the same way), the pitch and the usual car length.	Occupied spaces, even when lines are hidden under cars
A. Painted lines (OpenCV)	1) "paint" = pixels that are whitish (low colour saturation) and much brighter than the blurred background. Detected cars are blanked out first, because white cars are also "white and bright". 2) `cv2.HoughLinesP` finds straight lines (a large maxLineGap joins dashed lines). 3) With hints: only lines in the cars' directions are used, and two neighbouring lines form a space only if they are about one pitch apart (0.6-1.5x) and about one car long. Without cars (empty lot): the 3 strongest directions are tried, and a space must be 1.5-3.5 times longer than wide. 4) Pieces of the same painted line are merged first.	**Free** and occupied spaces - the only way to find free ones

Painted-line spaces are the most exact, so a car-based space is only added where no painted-line space exists (`merge_spaces`). A painted-line space is thrown away if it is far bigger/smaller than a car space, or if it cuts through a car's space without containing that car (`misaligned`). Right after creating the slots, one detection cycle marks them free/occupied with YOLO **and** the visual check, so a space over a car that YOLO missed (dark car, car under a tree) still becomes occupied.

Test result (crowded warm-filtered drone photo, 35 cars found): the 3 clearly empty spaces in the top row were found and shown free; spaces over 2 dark cars that YOLO missed were correctly occupied by the visual check. About 5 junk slots remained (across trees / between rows) and one wrong free slot - these can be deleted on Slot Setup.

Honest limits (found while testing):

- A "free" space can only be found where its white lines are clearly visible. Guessing free spaces from gaps between cars was tried and removed: in angled parking the row does not run along the cars' short side, so the guessed spaces came out crossed and tilted wrongly.
- Faded lines, warm colour filters (cream-looking paint) or very short lines may not be found - add those slots by hand.
- The pre-trained YOLO models do not recognise every car. In a very close top-down photo of one shiny black car, no model found it (the medium COCO model even called it a "cell phone"). The visual check (section 8.2b) still marks that space occupied, but the car is not counted in "Vehicles Seen". Training a custom model on a parking dataset (e.g. PKLot, CNRPark) would fix this - see section 15.

8.5 Why a background thread?

The AI must run all the time, even when no one has the website open. `detector.start()` starts a **daemon thread** (a second line of execution inside the same Python program) when FastAPI starts. The API answers requests on the main thread while the detector loop keeps working - this is the "autonomous agent" of the proposal.

9. Backend code explained

9.1 backend/requirements.txt

The list of Python libraries. `pip install -r requirements.txt` installs them all.

```
fastapi
uvicorn
python-multipart
pymysql
opencv-python
ultralytics
```

Library	Used for
fastapi	The web API framework
uvicorn	Runs the FastAPI app as a server
python-multipart	Needed by FastAPI to receive uploaded files
pymysql	Connects to MySQL
opencv-python	Camera reading, drawing, JPEG encoding
ultralytics	YOLOv8 (also installs PyTorch and NumPy)

9.2 backend/config.py

Every setting lives here so you never have to search the code to change something. Change `CAMERA_SOURCE` for the default camera, `DB_PASSWORD` if you set a MySQL password, and the AI thresholds to tune accuracy.

```
# ------------------------------------------------------------
# config.py - all settings of the backend in one place
# ------------------------------------------------------------

# MySQL settings (XAMPP default: user "root", empty password)
DB_HOST = "localhost"
DB_PORT = 3306
DB_USER = "root"
DB_PASSWORD = ""
DB_NAME = "smart_parking"

# Camera source:
#   "0"                         -> laptop / USB webcam
#   "uploads/parking.mp4"       -> a video file (loops forever)
#   "uploads/parking.jpg"       -> a single photo
#   "rtsp://user:pass@ip/..."   -> a real CCTV camera
CAMERA_SOURCE = "0"

# YOLO model (downloaded automatically the first time). Pick by camera angle:
#   "yolov8n-obb.pt" -> camera looking straight DOWN (drone / aerial photo)
#                       trained on the DOTA aerial dataset
#   "yolov8n.pt"     -> normal CCTV camera looking at the lot from an ANGLE
#                       trained on the COCO dataset
YOLO_MODEL = "yolov8n-obb.pt"
```

```
# Picture sizes YOLO may shrink the frame to. Cars look big (low camera) or
# small (high camera), so for every new source the AI tries each size once
# and keeps the one that finds the most vehicles.
IMAGE_SIZES = [160, 224, 320, 416, 640, 1024]

# Class names that count as a vehicle (covers both models above)
VEHICLE_NAMES = ["car", "motorcycle", "bus", "truck", "small vehicle", "large vehicle"]

# Minimum confidence for a detection to be accepted (0..1)
CONFIDENCE = 0.25

# A slot is "occupied" when a vehicle box covers at least this
# fraction of the slot area (0.30 = 30 %)
OCCUPIED_OVERLAP = 0.30

# Second opinion when YOLO misses a car: a slot is also "occupied" when its
# inside has much more light/dark contrast than the most even (empty) slot.
#   VISUAL_FACTOR 1.4 = at least 40 % more contrast than that empty slot
VISUAL_CHECK = True
VISUAL_FACTOR = 1.4

# How often the AI analyses a frame (seconds)
DETECT_INTERVAL = 1.0

# How often a row is saved into detection_logs (seconds)
LOG_INTERVAL = 10

# Folder where uploaded images / videos are stored
UPLOAD_DIR = "uploads"

# Websites allowed to call the API (CORS). "*" = any (fine for local XAMPP use)
ALLOWED_ORIGINS = ["*"]
```

9.3 backend/database.py

All SQL in one file. Every function opens a connection, runs one query and closes it again - simple and safe because the API and the AI thread never share a connection.

- `get_connection()` - connects with the settings from config.py. `DictCursor` returns each row as a Python dictionary (`row["slot_code"]`), which FastAPI turns into JSON directly. `autocommit=True` saves every change immediately.
- `init_database()` - creates the database and both tables if they are missing (`IF NOT EXISTS`), so the project works even without importing the .sql file.
- `get_slots / add_slot / delete_slot / delete_all_slots / update_slot_status` - CRUD for slots.
- `add_log / get_logs` - write and read history rows.
- The `%s` placeholders are filled by PyMySQL safely - this protects against **SQL injection**.
- Timestamps are converted with `str()` because JSON has no date type.

```
# ------------------------------------------------------------
# database.py - every MySQL query used by the project
# ------------------------------------------------------------
import json

import pymysql
import config


def get_connection(use_db=True):
```

```
    """Open a new connection to the XAMPP MySQL server."""
    return pymysql.connect(
        host=config.DB_HOST,
        port=config.DB_PORT,
        user=config.DB_USER,
        password=config.DB_PASSWORD,
        database=config.DB_NAME if use_db else None,
        cursorclass=pymysql.cursors.DictCursor,  # rows come back as dicts
        autocommit=True,
    )


def init_database():
    """Create the database and tables if they do not exist yet."""
    conn = get_connection(use_db=False)
    with conn.cursor() as cur:
        cur.execute(f"CREATE DATABASE IF NOT EXISTS {config.DB_NAME}")
    conn.close()

    conn = get_connection()
    with conn.cursor() as cur:
        cur.execute("""
            CREATE TABLE IF NOT EXISTS slots (
              id INT AUTO_INCREMENT PRIMARY KEY,
              slot_code VARCHAR(20) NOT NULL UNIQUE,
              x1 INT NOT NULL, y1 INT NOT NULL,
              x2 INT NOT NULL, y2 INT NOT NULL,
              points TEXT NULL,
              status ENUM('free','occupied') NOT NULL DEFAULT 'free',
              updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
                         ON UPDATE CURRENT_TIMESTAMP
            )""")
        cur.execute("""
            CREATE TABLE IF NOT EXISTS detection_logs (
              id INT AUTO_INCREMENT PRIMARY KEY,
              total_slots INT NOT NULL,
              free_slots INT NOT NULL,
              occupied_slots INT NOT NULL,
              cars_detected INT NOT NULL,
              cars_outside_slots INT NOT NULL,
              created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
            )""")
        # databases created by the first version have no "points" column -> add it
        cur.execute("SHOW COLUMNS FROM slots LIKE 'points'")
        if not cur.fetchone():
            cur.execute("ALTER TABLE slots ADD COLUMN points TEXT NULL AFTER y2")
    conn.close()


# ---------------- slots ----------------

def get_slots():
    conn = get_connection()
    with conn.cursor() as cur:
        cur.execute("SELECT * FROM slots ORDER BY slot_code")
        rows = cur.fetchall()
    conn.close()
    for row in rows:
        row["updated_at"] = str(row["updated_at"])  # make the timestamp JSON friendly
        # points are stored as text '[[x,y],[x,y],[x,y],[x,y]]' -> Python list
        row["points"] = json.loads(row["points"]) if row["points"] else None
    return rows


def add_slot(slot_code, x1, y1, x2, y2, points=None):
    """points = 4 corners [[x,y],...] for a tilted slot, or None for a rectangle."""
```

```
    conn = get_connection()
    with conn.cursor() as cur:
        cur.execute(
            "INSERT INTO slots (slot_code, x1, y1, x2, y2, points) VALUES (%s,%s,%s,%s,%s,%s)",
            (slot_code, x1, y1, x2, y2, json.dumps(points) if points else None),
        )
        new_id = cur.lastrowid
    conn.close()
    return new_id


def delete_slot(slot_id):
    conn = get_connection()
    with conn.cursor() as cur:
        cur.execute("DELETE FROM slots WHERE id = %s", (slot_id,))
    conn.close()


def delete_all_slots():
    conn = get_connection()
    with conn.cursor() as cur:
        cur.execute("DELETE FROM slots")
    conn.close()


def update_slot_status(slot_id, status):
    conn = get_connection()
    with conn.cursor() as cur:
        cur.execute("UPDATE slots SET status = %s WHERE id = %s", (status, slot_id))
    conn.close()


# ---------------- detection logs ----------------

def add_log(total, free, occupied, cars, outside):
    conn = get_connection()
    with conn.cursor() as cur:
        cur.execute(
            """INSERT INTO detection_logs
               (total_slots, free_slots, occupied_slots, cars_detected, cars_outside_slots)
               VALUES (%s,%s,%s,%s,%s)""",
            (total, free, occupied, cars, outside),
        )
    conn.close()


def get_logs(limit=20):
    conn = get_connection()
    with conn.cursor() as cur:
        cur.execute("SELECT * FROM detection_logs ORDER BY id DESC LIMIT %s", (limit,))
        rows = cur.fetchall()
    conn.close()
    for row in rows:
        row["created_at"] = str(row["created_at"])
    return rows
```

9.4 backend/detector.py - the AI agent

- `rect_polygon()` turns x1,y1,x2,y2 into 4 corners; `slot_polygon()` returns a slot's clicked corners or its rectangle corners.
- `overlap_ratio(slot_poly, car_poly)` - the formula from section 8.2, using OpenCV's polygon intersection so tilted slots and rotated cars work.

- `slot_contrast(gray, poly)` - the visual check of section 8.2b: cuts out the slot's inner 70 % (`fillPoly` mask), blurs it, and returns the brightness standard deviation.
- `choose_image_size()` - see "Automatic image size" in section 8.1. `set_source()` resets it so every new photo/video/camera is measured again.
- `class SlotDetector` - holds the model, the camera, the latest pictures and the latest numbers.
- `set_source()` - switches camera. It closes the old `VideoCapture`; the next `read_frame()` opens the new one. A `threading.Lock` prevents the API thread and AI thread from touching the camera at the same moment.
- `read_frame()` - photo → `cv2.imread`; webcam index ("0") → `cv2.VideoCapture(0)`; file or URL → `cv2.VideoCapture(path)`. When a video ends it jumps back to frame 0 so the demo never stops.
- `run()` also builds `vehicle_ids`: the class numbers of the loaded model whose names are in `VEHICLE_NAMES` (e.g. [9, 10] for the aerial model, [2, 3, 5, 7] for the COCO model).
- `detect_vehicles(frame, size)` - runs YOLO with those classes, the given size and the confidence limit. Aerial models put results in `result.obb` (`.xyxyxyxy` = 4 corners of each rotated box), normal models in `result.boxes` (upright boxes, turned into 4 corners). Returns a list of (polygon, confidence).
- Drawing uses `scale` (picture width / 640) so lines and text stay visible on very large photos.
- `process_frame()` - steps 3 to 8 of section 2.2. Note it only writes to MySQL **when a status changes**, so the database is not flooded and `updated_at` shows the real time of the change. Colours in OpenCV are BGR, so `(0,0,255)` is red.
- `run()` - loads the model once, then loops forever. The `try/except` means a broken camera or database hiccup never kills the agent; the error is shown on the dashboard instead.
- `detector = SlotDetector()` - one shared object that main.py imports.

```
# ------------------------------------------------------------
# detector.py - the AI agent (YOLO + OpenCV)
#
# Runs forever in a background thread:
#   1. read a frame from the camera / video / image
#   2. YOLO finds every vehicle in the frame
#   3. each vehicle shape is compared with every slot shape
#   4. slot status (free / occupied) is written to MySQL
#   5. an annotated picture is kept in memory for the website
# ------------------------------------------------------------
import os
import threading
import time

import cv2
import numpy as np
from ultralytics import YOLO

import config
import database
from auto_slots import find_car_spaces, find_painted_spaces, merge_spaces, sort_spaces

IMAGE_EXTENSIONS = (".jpg", ".jpeg", ".png", ".bmp", ".webp")


def rect_polygon(x1, y1, x2, y2):
    """The 4 corners of an upright rectangle."""
    return np.array([[x1, y1], [x2, y1], [x2, y2], [x1, y2]], dtype=np.float32)


def slot_polygon(slot):
    """A slot is either 4 clicked corners (tilted) or a dragged rectangle."""
```

```
    if slot.get("points"):
        return np.array(slot["points"], dtype=np.float32)
    return rect_polygon(slot["x1"], slot["y1"], slot["x2"], slot["y2"])


def overlap_ratio(slot_poly, car_poly):
    """How much of the slot area is covered by the vehicle (0..1)."""
    slot_area = cv2.contourArea(slot_poly)
    if slot_area <= 0:
        return 0.0
    intersection, _ = cv2.intersectConvexConvex(slot_poly, car_poly)
    return intersection / slot_area


def slot_contrast(gray, poly):
    """How 'busy' the inside of a slot looks. Bare ground is even (low value);
    a car has a dark body, windows and shadows (high value).
    Only the inner 70 % is used, so painted lines and kerbs at the edge don't count."""
    centre = poly.mean(axis=0)
    inner = (centre + (poly - centre) * 0.7).astype(np.int32)
    x, y, w, h = cv2.boundingRect(inner)
    x, y = max(x, 0), max(y, 0)
    roi = gray[y:y + h, x:x + w]
    if roi.size == 0:
        return 0.0
    mask = np.zeros(roi.shape, np.uint8)
    cv2.fillPoly(mask, [inner - [x, y]], 255)
    # blur away fine texture (bricks, asphalt grain) before measuring
    smooth = cv2.GaussianBlur(roi, (0, 0), max(2, min(w, h) / 40))
    return float(smooth[mask > 0].std())


class SlotDetector:
    def __init__(self):
        self.model = None
        self.vehicle_ids = []
        self.image_size = None      # chosen automatically for each new source
        self.source = config.CAMERA_SOURCE
        self.capture = None
        self.lock = threading.Lock()
        self.model_lock = threading.Lock()  # YOLO must not run in 2 threads at once
        self.running = False
        self.error = ""
        self.raw_jpeg = None        # last frame without drawings
        self.annotated_jpeg = None  # last frame with boxes drawn
        self.frame_width = 0
        self.frame_height = 0
        self.stats = {
            "total_slots": 0, "free_slots": 0, "occupied_slots": 0,
            "cars_detected": 0, "cars_outside_slots": 0, "last_update": None,
        }
        self.last_log_time = 0

    # ---------- camera handling ----------

    def set_source(self, source):
        """Switch to another camera / video / image."""
        with self.lock:
            self.source = str(source)
            if self.capture is not None:
                self.capture.release()
                self.capture = None
            self.image_size = None  # new picture -> choose the best size again
            self.error = ""

    def is_image_source(self):
```

```
        return self.source.lower().endswith(IMAGE_EXTENSIONS)

    def read_frame(self):
        """Return one frame (numpy image) or None."""
        with self.lock:
            if self.is_image_source():
                frame = cv2.imread(self.source)
                if frame is None:
                    self.error = f"Cannot open image: {self.source}"
                return frame

            if self.capture is None:
                # "0", "1" ... means a webcam index, anything else is a path / URL
                src = int(self.source) if self.source.isdigit() else self.source
                self.capture = cv2.VideoCapture(src)

            ok, frame = self.capture.read()
            if not ok and os.path.isfile(self.source):
                # video file finished -> start again from the first frame
                self.capture.set(cv2.CAP_PROP_POS_FRAMES, 0)
                ok, frame = self.capture.read()
            if not ok:
                self.error = f"Cannot read from camera source: {self.source}"
                return None
            self.error = ""
            return frame

    # ---------- AI part ----------

    def detect_vehicles(self, frame, size):
        """Run YOLO and return a list of (4-corner polygon, confidence)."""
        with self.model_lock:
            result = self.model(frame, classes=self.vehicle_ids, imgsz=size,
                                conf=config.CONFIDENCE, verbose=False)[0]
        if result.obb is not None:
            # aerial model: rotated boxes, already 4 corners each
            polygons = result.obb.xyxyxyxy.tolist()
            confs = result.obb.conf.tolist()
        else:
            # normal model: upright boxes -> turn into 4 corners
            polygons = [rect_polygon(*box) for box in result.boxes.xyxy.tolist()]
            confs = result.boxes.conf.tolist()
        return [(np.array(p, dtype=np.float32), c) for p, c in zip(polygons, confs)]

    def choose_image_size(self, frame):
        """Cars look big or small depending on how high the camera is.
        Try every size in IMAGE_SIZES once and keep the one that finds most vehicles."""
        counts = {size: len(self.detect_vehicles(frame, size)) for size in config.IMAGE_SIZES}
        self.image_size = max(counts, key=counts.get)
        print(f"[AI] vehicles found per size {counts} -> using {self.image_size}")

    def auto_create_slots(self):
        """Find the parking spaces in the current picture and save them as slots
        (replaces all existing slots). Returns what was found."""
        if self.model is None:
            raise RuntimeError("The AI model is still loading - try again in a few seconds")
        frame = self.read_frame()
        if frame is None:
            raise RuntimeError(self.error or "No camera picture yet")
        if self.image_size is None:
            self.choose_image_size(frame)
        cars = [poly for poly, _ in self.detect_vehicles(frame, self.image_size)]
        # method B: one space per parked car (+ line direction and spacing hints)
        car_spaces, hints = find_car_spaces(cars)
        # method A: white painted lines, searched in the directions the cars point
        line_spaces = find_painted_spaces(frame, hints, cars)
```

```
        spaces = merge_spaces(line_spaces, car_spaces)

        height, width = frame.shape[:2]
        database.delete_all_slots()
        for number, poly in enumerate(sort_spaces(spaces), start=1):
            # keep corners inside the picture
            pts = np.clip(poly, [0, 0], [width - 1, height - 1]).round().astype(int).tolist()
            xs, ys = [p[0] for p in pts], [p[1] for p in pts]
            database.add_slot(f"P{number}", min(xs), min(ys), max(xs), max(ys), pts)
        print(f"[AI] auto slots: {len(line_spaces)} from painted lines, "
              f"{len(spaces) - len(line_spaces)} from cars -> {len(spaces)} spaces")
        self.process_frame(frame)  # mark them free / occupied straight away
        return {"cars": len(cars), "slots": len(spaces), "free": self.stats["free_slots"]}

    def process_frame(self, frame):
        if self.image_size is None:
            self.choose_image_size(frame)
        cars = self.detect_vehicles(frame, self.image_size)
        slots = database.get_slots()
        slot_polys = [slot_polygon(s) for s in slots]

        # 1) second opinion without YOLO: compare every slot's contrast with the
        #    most even-looking slot in this picture (that one is almost surely empty)
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        contrasts = [slot_contrast(gray, p) for p in slot_polys]
        reference = min(contrasts, default=0.0)
        busy_limit = max(reference * config.VISUAL_FACTOR, reference + 8)

        # 2) decide the status of every slot
        free = 0
        for slot, poly, contrast in zip(slots, slot_polys, contrasts):
            best = max((overlap_ratio(poly, car) for car, _ in cars), default=0.0)
            seen_by_yolo = best >= config.OCCUPIED_OVERLAP
            looks_busy = config.VISUAL_CHECK and len(slots) > 1 and contrast > busy_limit
            new_status = "occupied" if seen_by_yolo or looks_busy else "free"
            if new_status == "free":
                free += 1
            if new_status != slot["status"]:          # only write changes
                database.update_slot_status(slot["id"], new_status)
            slot["status"] = new_status

        # 3) vehicles that are not inside any slot = anomaly
        outside = 0
        for car, _ in cars:
            if all(overlap_ratio(p, car) < config.OCCUPIED_OVERLAP for p in slot_polys):
                outside += 1

        # 4) draw the result on a copy of the frame
        #    (line width and text size grow with the picture so they stay visible)
        annotated = frame.copy()
        scale = max(1, frame.shape[1] // 640)
        for car, _ in cars:
            cv2.polylines(annotated, [car.astype(np.int32)], True, (0, 165, 255), 2 * scale)
        for slot, poly in zip(slots, slot_polys):
            color = (0, 0, 255) if slot["status"] == "occupied" else (0, 200, 0)  # BGR
            cv2.polylines(annotated, [poly.astype(np.int32)], True, color, 3 * scale)
            cv2.putText(annotated, slot["slot_code"],
                        (slot["x1"] + 5 * scale, slot["y1"] + 25 * scale),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7 * scale, color, 2 * scale)

        # 5) save results in memory for the API
        total = len(slots)
        self.raw_jpeg = cv2.imencode(".jpg", frame)[1].tobytes()
        self.annotated_jpeg = cv2.imencode(".jpg", annotated)[1].tobytes()
        self.frame_height, self.frame_width = frame.shape[:2]
        self.stats = {
```

```
            "total_slots": total,
            "free_slots": free,
            "occupied_slots": total - free,
            "cars_detected": len(cars),
            "cars_outside_slots": outside,
            "last_update": time.strftime("%Y-%m-%d %H:%M:%S"),
        }

        # 6) save a history row every LOG_INTERVAL seconds
        if time.time() - self.last_log_time >= config.LOG_INTERVAL:
            database.add_log(total, free, total - free, len(cars), outside)
            self.last_log_time = time.time()

    # ---------- main loop ----------

    def run(self):
        print("[AI] loading YOLO model ...")
        self.model = YOLO(config.YOLO_MODEL)
        # class ids of this model whose name is a vehicle, e.g. {2: "car"} -> [2]
        self.vehicle_ids = [i for i, name in self.model.names.items()
                            if name in config.VEHICLE_NAMES]
        print("[AI] model ready, watching camera source:", self.source)
        self.running = True
        while self.running:
            try:
                frame = self.read_frame()
                if frame is not None:
                    self.process_frame(frame)
            except Exception as e:  # keep the agent alive whatever happens
                self.error = str(e)
                print("[AI] error:", e)
            time.sleep(config.DETECT_INTERVAL)

    def start(self):
        threading.Thread(target=self.run, daemon=True).start()

    def stop(self):
        self.running = False


# one shared detector object used by main.py
detector = SlotDetector()
```

9.5 backend/auto_slots.py - automatic parking-space detection

The two methods of section 8.4. Helper functions:

- `car_geometry()` - from a car's 4 corners: centre, direction of its short side (= direction of the row), width and length.
- `space_polygon()` - builds a turned rectangle (4 corners) from a centre, direction, width and length.
- `find_car_spaces()` - method B. Finds each car's neighbour in the same row (similar angle, beside it, not in front of it) to measure the usual spacing ("pitch"), makes one space per car, and returns the hints (car directions, pitch, car length).
- `find_painted_spaces(frame, hints, car_polys)` + `spaces_between_lines()` - method A. Works on a 1000-pixel copy for speed and scales the result back to full size.
- `misaligned()` - true if a line-space cuts through a car's space without containing that car's centre.
- `remove_inner_spaces()` - drops a space whose centre lies inside a bigger one (duplicate).
- `merge_spaces()` and `sort_spaces()` - combine both methods and order the result for the names P1, P2, ...

```
# ------------------------------------------------------------
# auto_slots.py - find parking spaces automatically
#
# Two methods are combined:
# A) PAINTED LINES (OpenCV): white stall lines are found with the
#    Hough transform; the area between two neighbouring parallel
#    lines is one parking space. Works even when the lot is empty.
# B) PARKED CARS (YOLO): cars stand side by side in rows with about
#    the same distance between them (the "pitch").
#      * every detected car -> one parking space
#      * a gap in a row as wide as 1 or more pitches -> that many
#        more spaces
#    Gap spaces are NOT assumed to be free: the detector decides
#    their status like any slot (YOLO + visual check), so a car that
#    YOLO missed (e.g. under a tree) still makes its space occupied.
# ------------------------------------------------------------
import cv2
import numpy as np


def car_geometry(poly):
    """centre, unit vector along the row (= car's short side), car width, car length."""
    p = np.asarray(poly, dtype=np.float32)
    centre = p.mean(axis=0)
    edge1, edge2 = p[1] - p[0], p[2] - p[1]
    len1, len2 = float(np.linalg.norm(edge1)), float(np.linalg.norm(edge2))
    if len1 < len2:
        return centre, edge1 / len1, len1, len2
    return centre, edge2 / len2, len2, len1


def space_polygon(centre, row_dir, width, length):
    """4 corners of a parking space centred on 'centre', turned like the row."""
    across = np.array([-row_dir[1], row_dir[0]])  # along the car's length
    w, l = row_dir * width / 2, across * length / 2
    return np.array([centre - w - l, centre + w - l, centre + w + l, centre - w + l],
                    dtype=np.float32)


# ---------------- method B: parked cars ----------------

def find_car_spaces(car_polys):
    """A space around every detected car.
    Also returns 'hints' for method A: the directions the parked cars point in
    (= direction of the painted stall lines), the usual space width (pitch) and
    the usual car length."""
    cars = [car_geometry(p) for p in car_polys]
    if not cars:
        return [], None

    # 1) for every car, find the next car beside it in the same row
    pairs = []  # (car i, car j, distance between them along the row)
    for i, (c_i, dir_i, w_i, l_i) in enumerate(cars):
        side = np.array([-dir_i[1], dir_i[0]])
        best = None
        for j, (c_j, dir_j, _, _) in enumerate(cars):
            if j == i or abs(float(dir_i @ dir_j)) < 0.9:
                continue  # same car, or parked at another angle -> another row
            along = float((c_j - c_i) @ dir_i)        # distance along the row
            sideways = abs(float((c_j - c_i) @ side))  # distance out of the row
            if along > 0 and sideways < 0.25 * l_i and (best is None or along < best[1]):
                best = (j, along)
        if best:
            pairs.append((i, best[0], best[1]))
```

```
    # 2) pitch = usual distance between two neighbouring cars = width of a space
    width = float(np.median([w for _, _, w, _ in cars]))
    close = [d for _, _, d in pairs if d < 1.6 * width]
    pitch = float(np.median(close)) if close else 1.25 * width

    # 3) a space around every car, 90 % of the pitch wide so neighbours don't touch
    spaces = [space_polygon(c, d, pitch * 0.9, l) for c, d, _, l in cars]

    # 4) hints: the different directions cars are parked in (lines run the same way)
    angles = []
    for _, d, _, _ in cars:
        angle = np.degrees(np.arctan2(d[0], -d[1])) % 180  # direction of the car's length
        if all(abs((angle - a + 90) % 180 - 90) > 8 for a in angles):
            angles.append(angle)
    hints = {"angles": angles, "pitch": pitch,
             "length": float(np.median([l for _, _, _, l in cars]))}
    return spaces, hints


# ---------------- method A: painted lines ----------------

def find_painted_spaces(frame, hints=None, car_polys=()):
    """Find parking spaces between parallel white painted lines.
    hints (from method B) say in which directions the stall lines run, how far
    apart they are and how long a space is; without them the directions are guessed.
    Returns a list of 4-corner polygons in full-picture pixels."""
    scale = 1000 / max(frame.shape[:2])     # work on a ~1000 px copy (fast, same result)
    small = cv2.resize(frame, None, fx=scale, fy=scale)
    gray = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY)
    saturation = cv2.cvtColor(small, cv2.COLOR_BGR2HSV)[:, :, 1]
    size = max(gray.shape)
    car_angles = hints["angles"] if hints else None
    pitch_small = hints["pitch"] * scale if hints else None
    length_small = hints["length"] * scale if hints else None

    # 1) paint = WHITE pixels (little colour) that are much brighter than their surroundings.
    #    When the cars tell us the line direction, junk is filtered later, so slightly
    #    cream / yellowed paint (warm photo filters) is allowed too.
    max_saturation, min_brighter, votes = (90, 30, 30) if car_angles else (50, 35, 40)
    background = cv2.GaussianBlur(gray, (0, 0), 15)
    paint = (((gray.astype(int) - background) > min_brighter) & (saturation < max_saturation))
    paint = paint.astype(np.uint8) * 255
    # white / silver cars are also "white and bright" -> blank out every detected car
    for poly in car_polys:
        cv2.fillPoly(paint, [(np.asarray(poly) * scale).astype(np.int32)], 0)

    # 2) straight lines (maxLineGap joins the pieces of dashed lines)
    min_length = length_small * 0.5 if length_small else size * 0.08
    found = cv2.HoughLinesP(paint, 1, np.pi / 180, threshold=votes,
                            minLineLength=int(min_length), maxLineGap=int(size * 0.03))
    if found is None:
        return []
    lines = []
    for x1, y1, x2, y2 in found.reshape(-1, 4).astype(float):
        angle = np.degrees(np.arctan2(y2 - y1, x2 - x1)) % 180
        lines.append((angle, np.array([x1, y1], float), np.array([x2, y2], float)))

    # 3a) directions known from the parked cars -> look for spaces in all of them
    if car_angles:
        spaces = []
        for angle in car_angles:
            spaces += spaces_between_lines(lines, angle, size, pitch_small, length_small)
        return [s / scale for s in remove_inner_spaces(spaces)]  # back to full-size pixels

    # 3b) no cars: strongest line directions (kerbs, stall lines, car edges ...)
    weights = np.zeros(180)
```

```
    for angle, p1, p2 in lines:
        weights[int(angle) % 180] += np.linalg.norm(p2 - p1)
    weights = sum(np.roll(weights, k) for k in (-2, -1, 0, 1, 2))  # smooth +-2 degrees
    candidates = []
    for _ in range(3):
        best = int(np.argmax(weights))
        if weights[best] == 0:
            break
        candidates.append(best)
        for k in range(-10, 11):  # ignore directions close to this one next time
            weights[(best + k) % 180] = 0

    # 4) the direction that gives the most parking spaces is the stall-line direction
    best_spaces = max((spaces_between_lines(lines, a, size) for a in candidates), key=len)
    return [s / scale for s in best_spaces]  # back to full-size pixels


def spaces_between_lines(lines, angle_deg, size, pitch=None, car_length=None):
    """Parking spaces formed by the lines that run in direction angle_deg.
    pitch / car_length (if known) = expected width / length of a space."""
    angle = np.radians(angle_deg)
    along = np.array([np.cos(angle), np.sin(angle)])
    across = np.array([-along[1], along[0]])

    # describe each line by its position across the stalls (offset)
    # and where it starts / ends along itself (t1, t2)
    stall_lines = []
    for a, p1, p2 in lines:
        if abs((a - angle_deg + 90) % 180 - 90) <= 8:
            t = sorted([p1 @ along, p2 @ along])
            stall_lines.append([float((p1 + p2) @ across / 2), t[0], t[1]])
    stall_lines.sort()

    # pieces of the same painted line (close offset, overlapping) -> one line
    merged = []
    for offset, t1, t2 in stall_lines:
        for m in merged:
            if abs(offset - m[0]) < size * 0.012 and t1 <= m[2] + size * 0.03 and t2 >= m[1] - size *
0.03:
                m[0] = (m[0] + offset) / 2
                m[1], m[2] = min(m[1], t1), max(m[2], t2)
                break
        else:
            merged.append([offset, t1, t2])
    merged.sort()

    # each line + the next line beside it (that runs next to it) = one parking space
    spaces = []
    for i, (o1, a1, a2) in enumerate(merged):
        for o2, b1, b2 in merged[i + 1:]:
            start, end = max(a1, b1), min(a2, b2)  # part where both lines exist
            if end - start < 0.6 * max(a2 - a1, b2 - b1):
                continue  # not side by side, or much shorter (another row, edge of a car ...)
            width, length = o2 - o1, end - start
            if pitch:
                # we know how wide a space is: the next line must be about one pitch away
                if width < 0.6 * pitch:
                    continue  # too close: edge of a car, double line ...
                # ... and a space is about as long as a car
                is_space = width <= 1.5 * pitch and 0.6 * car_length <= length <= 1.5 * car_length
            else:
                # a real parking space is about 1.5 - 3.5 times longer than it is wide
                if length / width > 3.5:
                    continue  # too narrow: probably not a stall line (e.g. edge of a car)
                # not a tiny shape, and not bigger than a parking space can be in a photo
                is_space = length / width >= 1.5 and size * 0.025 <= width <= size * 0.25
```

```
            if is_space:
                spaces.append(np.array([o1 * across + start * along, o2 * across + start * along,
                                        o2 * across + end * along, o1 * across + end * along],
                                       dtype=np.float32))
            break  # only the nearest real neighbouring line
    return remove_inner_spaces(spaces)


def remove_inner_spaces(spaces):
    """Drop a space whose centre lies inside another, bigger space (duplicate)."""
    keep = []
    for s in spaces:
        centre = tuple(float(v) for v in s.mean(axis=0))
        inside_bigger = any(
            cv2.contourArea(o) > cv2.contourArea(s) and cv2.pointPolygonTest(o, centre, False) >= 0
            for o in spaces if o is not s)
        if not inside_bigger:
            keep.append(s)
    return keep


def misaligned(space, car_spaces):
    """True if the space cuts through a car's space without holding that car
    (a real space around a car contains the car's centre)."""
    area = cv2.contourArea(space)
    for car in car_spaces:
        overlap, _ = cv2.intersectConvexConvex(space, car)
        centre = tuple(float(v) for v in car.mean(axis=0))
        if overlap > 0.25 * area and cv2.pointPolygonTest(space, centre, False) < 0:
            return True
    return False


def merge_spaces(line_spaces, car_spaces):
    """Painted-line spaces are the most exact. Add a car-based space only where
    no painted-line space exists yet."""
    if car_spaces:
        # a painted-line "space" much bigger or smaller than a car space is not a
        # parking space (e.g. two long kerb lines) -> drop it
        typical = float(np.median([cv2.contourArea(s) for s in car_spaces]))
        line_spaces = [s for s in line_spaces
                       if 0.4 * typical <= cv2.contourArea(s) <= 2 * typical
                       and not misaligned(s, car_spaces)]
    result = list(line_spaces)
    for space in car_spaces:
        centre = space.mean(axis=0)
        width = min(np.linalg.norm(space[1] - space[0]), np.linalg.norm(space[2] - space[1]))
        if all(np.linalg.norm(centre - s.mean(axis=0)) > 0.6 * width for s in result):
            result.append(space)
    return result


def sort_spaces(spaces):
    """Order spaces top-to-bottom by row, then left-to-right (for names P1, P2, ...)."""
    if not spaces:
        return []
    row_height = float(np.median([np.ptp(s[:, 1]) for s in spaces]))
    return sorted(spaces, key=lambda s: (round(s[:, 1].mean() / row_height), s[:, 0].mean()))
```

In `detector.py`, `auto_create_slots()` uses these functions, replaces all slots in MySQL, and runs one detection cycle. A `model_lock` makes sure YOLO never runs in two threads at the same time (the API thread and the AI thread).

9.6 backend/main.py - the REST API

- `lifespan()` - code before `yield` runs when the server starts (create tables, create the uploads folder, start the AI thread); code after runs when it stops.
- `CORSMiddleware` - the website comes from Apache (`http://localhost`) but the API is on `http://localhost:8000`, a different "origin". Browsers block such calls unless the API allows them; this middleware allows them.
- `SlotIn`, `SourceIn` - Pydantic models. FastAPI checks that the JSON body has the right fields and types and returns error 422 automatically if not.
- `create_slot` handles both shapes. For a rectangle it sorts the coordinates (the user may drag from bottom-right to top-left). For 4 corners, `sort_corners()` orders them by angle around their centre (`math.atan2`) so the shape never crosses itself, whatever order they were clicked, and x1..y2 becomes the box around them. Tiny slots are rejected and a duplicate code becomes a friendly 400 error.
- `get_frame` returns the latest JPEG straight from memory - no files are written, so it is fast.
- `upload_source` checks the file extension, saves it as `uploads/source.ext` and tells the detector to use it.

```
# ------------------------------------------------------------
# main.py - FastAPI web server (REST API)
# Start with:   uvicorn main:app --reload --port 8000
# API docs:     http://localhost:8000/docs
# ------------------------------------------------------------
import math
import os
import shutil
from contextlib import asynccontextmanager

import pymysql
from fastapi import FastAPI, File, HTTPException, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import Response
from pydantic import BaseModel

import config
import database
from detector import detector


@asynccontextmanager
async def lifespan(app: FastAPI):
    # runs once when the server starts
    database.init_database()
    os.makedirs(config.UPLOAD_DIR, exist_ok=True)
    detector.start()
    yield
    # runs once when the server stops
    detector.stop()


app = FastAPI(title="Smart Parking API", lifespan=lifespan)

# allow the React website (served by XAMPP Apache) to call this API
app.add_middleware(
    CORSMiddleware,
    allow_origins=config.ALLOWED_ORIGINS,
    allow_methods=["*"],
    allow_headers=["*"],
```

```
)


# ---------- request bodies ----------

class SlotIn(BaseModel):
    slot_code: str
    x1: int
    y1: int
    x2: int
    y2: int
    points: list[list[int]] | None = None  # 4 corners for a tilted slot


class SourceIn(BaseModel):
    source: str


# ---------- endpoints ----------

@app.get("/")
def home():
    return {"message": "Smart Parking API is running. Open /docs for details."}


@app.get("/api/status")
def get_status():
    """Live numbers shown on the dashboard."""
    return {
        **detector.stats,
        "source": detector.source,
        "ai_running": detector.running,
        "error": detector.error,
        "frame_width": detector.frame_width,
        "frame_height": detector.frame_height,
    }


@app.get("/api/slots")
def list_slots():
    return database.get_slots()


def sort_corners(points):
    """Put 4 clicked corners in order around their centre, so the shape
    never crosses itself, whatever order the user clicked them in."""
    cx = sum(p[0] for p in points) / 4
    cy = sum(p[1] for p in points) / 4
    return sorted(points, key=lambda p: math.atan2(p[1] - cy, p[0] - cx))


@app.post("/api/slots")
def create_slot(slot: SlotIn):
    points = None
    if slot.points:
        if len(slot.points) != 4:
            raise HTTPException(400, "A tilted slot needs exactly 4 corners")
        points = sort_corners(slot.points)
        xs = [p[0] for p in points]
        ys = [p[1] for p in points]
        x1, y1, x2, y2 = min(xs), min(ys), max(xs), max(ys)  # box around the corners
    else:
        # make sure (x1,y1) is top-left and (x2,y2) is bottom-right
        x1, x2 = sorted([slot.x1, slot.x2])
        y1, y2 = sorted([slot.y1, slot.y2])
    if x2 - x1 < 10 or y2 - y1 < 10:
```

```
        raise HTTPException(400, "Slot is too small")
    try:
        new_id = database.add_slot(slot.slot_code.strip(), x1, y1, x2, y2, points)
    except pymysql.err.IntegrityError:
        raise HTTPException(400, f"Slot code '{slot.slot_code}' already exists")
    return {"id": new_id}


@app.post("/api/slots/auto")
def auto_slots():
    """Let the AI find all parking spaces in the current picture (replaces all slots)."""
    try:
        return detector.auto_create_slots()
    except RuntimeError as e:
        raise HTTPException(400, str(e))


@app.delete("/api/slots/{slot_id}")
def remove_slot(slot_id: int):
    database.delete_slot(slot_id)
    return {"deleted": slot_id}


@app.delete("/api/slots")
def remove_all_slots():
    database.delete_all_slots()
    return {"deleted": "all"}


@app.get("/api/frame")
def get_frame(annotated: bool = True):
    """Latest camera picture as JPEG (annotated = with boxes drawn)."""
    data = detector.annotated_jpeg if annotated else detector.raw_jpeg
    if data is None:
        raise HTTPException(404, "No frame yet")
    return Response(content=data, media_type="image/jpeg",
                    headers={"Cache-Control": "no-store"})


@app.post("/api/source")
def change_source(body: SourceIn):
    """Use a webcam index ("0"), a file path or an RTSP/HTTP camera URL."""
    detector.set_source(body.source.strip())
    return {"source": detector.source}


@app.post("/api/source/upload")
def upload_source(file: UploadFile = File(...)):
    """Upload a parking-lot photo or video and start analysing it."""
    ext = os.path.splitext(file.filename)[1].lower()
    if ext not in (".jpg", ".jpeg", ".png", ".bmp", ".webp", ".mp4", ".avi", ".mov", ".mkv"):
        raise HTTPException(400, "Please upload an image or a video file")
    path = os.path.join(config.UPLOAD_DIR, "source" + ext)
    with open(path, "wb") as out:
        shutil.copyfileobj(file.file, out)
    detector.set_source(path)
    # a new picture means new parking spaces -> find them automatically
    try:
        found = detector.auto_create_slots()
    except RuntimeError as e:
        found = {"error": str(e)}
    return {"source": path, **found}


@app.get("/api/logs")
```

```
def list_logs(limit: int = 20):
    return database.get_logs(limit)
```

9.7 backend/.htaccess and database/.htaccess

Because the project sits inside `htdocs`, Apache could show these files to anyone. `Require all denied` blocks web access to these folders (the same is done for `frontend/`).

10. REST API reference

Base URL: `http://localhost:8000` - interactive docs: `http://localhost:8000/docs`

Method	URL	Body	Returns
GET	/api/status	-	Counters, last update, current source, AI running, error, frame size
GET	/api/slots	-	All slots with status
POST	/api/slots	Rectangle: `{"slot_code":"A1","x1":10,"y1":20,"x2":110,"y2":220}` Tilted: add `"points":[[10,20],[90,5],[110,200],[30,215]]`	`{"id": 1}`
POST	/api/slots/auto	-	AI finds all spaces in the current picture (replaces all slots): `{"cars": 0, "slots": 3, "free": 3}`
DELETE	/api/slots/{id}	-	Deletes one slot
DELETE	/api/slots	-	Deletes all slots
GET	/api/frame?annotated=true	-	JPEG picture (false = without drawings)
POST	/api/source	`{"source":"0"}`	Switches the camera
POST	/api/source/upload	form-data `file`	Uploads a photo/video, uses it, and auto-detects its parking spaces
GET	/api/logs?limit=20	-	Latest history rows

Example `/api/status` answer:

```
{ "total_slots": 12, "free_slots": 5, "occupied_slots": 7, "cars_detected": 8,
  "cars_outside_slots": 1, "last_update": "2026-10-13 14:05:31", "source": "uploads\\source.mp4",
  "ai_running": true, "error": "", "frame_width": 1280, "frame_height": 720 }
```


11. Frontend code explained

The source is in `frontend/`. It is written in TypeScript (`.ts`) and TSX (`.tsx` = TypeScript + HTML-like JSX). Vite turns it into the plain files in `app/` that Apache serves.

11.1 Configuration files

frontend/package.json

Lists the JavaScript libraries and the commands: `npm run dev` (development server with live reload), `npm run build` (type-check with `tsc`, then build into `../app`).

```
{
  "name": "smart-parking-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc --noEmit && vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.1.0",
    "react-dom": "^19.1.0"
  },
  "devDependencies": {
    "@tailwindcss/vite": "^4.1.8",
    "@types/react": "^19.1.6",
    "@types/react-dom": "^19.1.5",
    "@vitejs/plugin-react": "^4.5.0",
    "tailwindcss": "^4.1.8",
    "typescript": "~5.8.3",
    "vite": "^6.3.5"
  }
}
```

frontend/vite.config.ts

Enables the React and Tailwind plugins. `base: "./"` makes all links relative so the site works inside `/smart-parking/app/`; `outDir: "../app"` puts the build straight into the XAMPP folder.

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
  base: "./", // relative paths so the site works inside any htdocs folder
  build: {
    outDir: "../app", // XAMPP serves it at http://localhost/smart-parking/app/
    emptyOutDir: true,
```

```
  },
});
```

frontend/tsconfig.json

TypeScript settings. `"strict": true` turns on all safety checks; `"jsx": "react-jsx"` lets us write JSX.

```
{
  "compilerOptions": {
    "target": "ES2020",
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true,
    "skipLibCheck": true,
    "noEmit": true,
    "isolatedModules": true
  },
  "include": ["src"]
}
```

frontend/index.html, src/main.tsx, src/index.css, src/vite-env.d.ts

`index.html` has an empty `<div id="root">`; `main.tsx` mounts the React `App` into it. `index.css` has one line that loads Tailwind. `vite-env.d.ts` gives TypeScript the Vite type definitions.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Smart Parking</title>
  </head>
  <body class="bg-slate-100">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App";
import "./index.css";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

```
@import "tailwindcss";
```

```
/// <reference types="vite/client" />
```

11.2 src/types.ts

TypeScript **interfaces** that describe the JSON returned by the backend. They match the MySQL columns and the `/api/status` answer exactly. `"free" | "occupied"` means status can only be one of those two strings.

```
// Shapes of the JSON data returned by the FastAPI backend

export interface Slot {
  id: number;
  slot_code: string;
  x1: number;
  y1: number;
  x2: number;
  y2: number;
  points: number[][] | null; // 4 corners of a tilted slot, null = rectangle
  status: "free" | "occupied";
  updated_at: string;
}

export interface Status {
  total_slots: number;
  free_slots: number;
  occupied_slots: number;
  cars_detected: number;
  cars_outside_slots: number;
  last_update: string | null;
  source: string;
  ai_running: boolean;
  error: string;
  frame_width: number;
  frame_height: number;
}

export interface Log {
  id: number;
  total_slots: number;
  free_slots: number;
  occupied_slots: number;
  cars_detected: number;
  cars_outside_slots: number;
  created_at: string;
}
```

11.3 src/api.ts

Every call to the backend is in this one file. `request<T>()` is a generic helper: it calls `fetch`, throws an error with FastAPI's `detail` message if the answer is not OK, and returns the JSON typed as `T`. `frameUrl()` adds `t=Date.now()` so the browser downloads a fresh picture every time instead of a cached one. To use another backend address, change `API_URL` and rebuild.

```
// All calls to the FastAPI backend live in this file
import type { Log, Slot, Status } from "./types";

export const API_URL = "http://localhost:8000";

async function request<T>(path: string, options?: RequestInit): Promise<T> {
  const res = await fetch(API_URL + path, options);
  if (!res.ok) {
```

```
    const body = await res.json().catch(() => ({}));
    throw new Error(body.detail || `Request failed (${res.status})`);
  }
  return res.json();
}

export const getStatus = () => request<Status>("/api/status");
export const getSlots = () => request<Slot[]>("/api/slots");
export const getLogs = () => request<Log[]>("/api/logs?limit=15");

export const addSlot = (slot: Omit<Slot, "id" | "status" | "updated_at">) =>
  request<{ id: number }>("/api/slots", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(slot),
  });

export const deleteSlot = (id: number) =>
  request(`/api/slots/${id}`, { method: "DELETE" });

export const deleteAllSlots = () => request("/api/slots", { method: "DELETE" });

// what the AI found when it searched the picture for parking spaces
export interface AutoResult {
  cars: number;
  slots: number;
  free: number;
  error?: string;
}

export const autoSlots = () => request<AutoResult>("/api/slots/auto", { method: "POST" });

export const setSource = (source: string) =>
  request("/api/source", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ source }),
  });

export const uploadSource = (file: File) => {
  const form = new FormData();
  form.append("file", file);
  return request<Partial<AutoResult>>("/api/source/upload", { method: "POST", body: form });
};

// "t" changes every call so the browser never shows a cached picture
export const frameUrl = (annotated: boolean) =>
  `${API_URL}/api/frame?annotated=${annotated}&t=${Date.now()}`;
```

11.4 src/App.tsx

The page frame: blue header with the title and two tabs. `useState<Page>` remembers which tab is open and shows either `Dashboard` or `SlotSetup` (no router needed).

```
import { useState } from "react";
import Dashboard from "./components/Dashboard";
import SlotSetup from "./components/SlotSetup";

type Page = "dashboard" | "setup";

export default function App() {
  const [page, setPage] = useState<Page>("dashboard");
```

```
  const tab = (name: Page, label: string) => (
    <button
      onClick={() => setPage(name)}
      className={`px-4 py-2 rounded-lg text-sm font-medium ${
        page === name ? "bg-white text-blue-700" : "text-blue-100 hover:bg-blue-600"
      }`}
    >
      {label}
    </button>
  );

  return (
    <div className="min-h-screen">
      <header className="bg-blue-700 text-white shadow">
        <div className="max-w-7xl mx-auto px-4 py-4 flex flex-wrap items-center justify-between gap-3">
          <div>
            <h1 className="text-xl font-bold">Smart Parking</h1>
            <p className="text-xs text-blue-100">Live Free-Slot Counting from Camera Feed - YOLO AI</p>
          </div>
          <nav className="flex gap-2">
            {tab("dashboard", "Live Dashboard")}
            {tab("setup", "Slot Setup")}
          </nav>
        </div>
      </header>

      <main className="max-w-7xl mx-auto px-4 py-6">
        {page === "dashboard" ? <Dashboard /> : <SlotSetup />}
      </main>
    </div>
  );
}
```

11.5 src/components/Dashboard.tsx

The live page. `useEffect` runs once when the page opens: it calls `load()` immediately and then every 2 s with `setInterval`. `Promise.all` fetches status, slots and logs at the same time. The returned function `clearInterval` stops the timer when you switch tabs. If the backend cannot be reached, a red help message is shown; if the AI reports an error (e.g. camera not found), a yellow message is shown.

```
import { useEffect, useState } from "react";
import { getLogs, getSlots, getStatus } from "../api";
import type { Log, Slot, Status } from "../types";
import LiveFeed from "./LiveFeed";
import LogsTable from "./LogsTable";
import SlotMap from "./SlotMap";
import SourcePanel from "./SourcePanel";
import StatCard from "./StatCard";

export default function Dashboard() {
  const [status, setStatus] = useState<Status | null>(null);
  const [slots, setSlots] = useState<Slot[]>([]);
  const [logs, setLogs] = useState<Log[]>([]);
  const [offline, setOffline] = useState(false);

  // ask the backend for fresh data every 2 seconds
  useEffect(() => {
    const load = async () => {
      try {
        const [st, sl, lg] = await Promise.all([getStatus(), getSlots(), getLogs()]);
        setStatus(st);
        setSlots(sl);
```

```
        setLogs(lg);
        setOffline(false);
      } catch {
        setOffline(true);
      }
    };
    load();
    const timer = setInterval(load, 2000);
    return () => clearInterval(timer);
  }, []);

  return (
    <div className="space-y-6">
      {offline && (
        <div className="bg-red-100 text-red-700 p-3 rounded-lg text-sm">
          Cannot reach the backend at http://localhost:8000. Start it with:
          <code className="ml-1 font-mono">uvicorn main:app --port 8000</code>
        </div>
      )}
      {status?.error && (
        <div className="bg-amber-100 text-amber-800 p-3 rounded-lg text-sm">AI: {status.error}</div>
      )}

      <div className="grid grid-cols-2 md:grid-cols-5 gap-4">
        <StatCard label="Total Slots" value={status?.total_slots ?? 0} color="text-slate-800" />
        <StatCard label="Free Slots" value={status?.free_slots ?? 0} color="text-green-600" />
        <StatCard label="Occupied" value={status?.occupied_slots ?? 0} color="text-red-600" />
        <StatCard label="Vehicles Seen" value={status?.cars_detected ?? 0} color="text-blue-600" />
        <StatCard label="Outside Slots" value={status?.cars_outside_slots ?? 0} color="text-amber-600"
/>
      </div>
      <p className="text-xs text-slate-500">
        AI agent: {status?.ai_running ? "running" : "starting..."} | last update:{" "}
        {status?.last_update ?? "-"}
      </p>

      <div className="grid lg:grid-cols-3 gap-6">
        <div className="lg:col-span-2">
          <LiveFeed />
        </div>
        <div className="space-y-6">
          <SourcePanel current={status?.source ?? ""} />
          <SlotMap slots={slots} />
        </div>
      </div>

      <LogsTable logs={logs} />
    </div>
  );
}
```

11.6 src/components/StatCard.tsx

A small reusable box with a label and a big number. The `Props` interface defines what it accepts.

```
interface Props {
  label: string;
  value: number | string;
  color: string; // Tailwind text colour class, e.g. "text-green-600"
}

export default function StatCard({ label, value, color }: Props) {
  return (
```

```
    <div className="bg-white rounded-xl shadow p-4">
      <p className="text-sm text-slate-500">{label}</p>
      <p className={`text-3xl font-bold ${color}`}>{value}</p>
    </div>
  );
}
```

11.7 src/components/LiveFeed.tsx

Shows the annotated JPEG from `/api/frame` and changes the image URL every second, which makes the browser reload it - a simple "live video" without streaming code. `onError` shows a waiting message until the first frame exists.

```
import { useEffect, useState } from "react";
import { frameUrl } from "../api";

// Shows the AI-annotated camera picture and refreshes it every second
export default function LiveFeed() {
  const [src, setSrc] = useState(frameUrl(true));
  const [failed, setFailed] = useState(false);

  useEffect(() => {
    const timer = setInterval(() => setSrc(frameUrl(true)), 1000);
    return () => clearInterval(timer);
  }, []);

  return (
    <div className="bg-white rounded-xl shadow p-4">
      <h2 className="font-semibold mb-3">Live Camera (AI view)</h2>
      {failed && (
        <p className="text-sm text-slate-500 mb-2">
          Waiting for the first frame from the AI... (is the backend running?)
        </p>
      )}
      <img
        src={src}
        alt="Live parking lot"
        className={`block mx-auto max-w-full max-h-[75vh] rounded-lg bg-slate-200 ${failed ? "hidden" :
""}`}
        onLoad={() => setFailed(false)}
        onError={() => setFailed(true)}
      />
      <p className="text-xs text-slate-500 mt-2">
        Green box = free slot, Red box = occupied slot, Orange box = vehicle detected by YOLO
      </p>
    </div>
  );
}
```

11.8 src/components/SlotMap.tsx

The colour map of the lot: one tile per slot, green when free and red when occupied. `.map()` turns the array of slots into an array of tiles; `key` helps React update only changed tiles.

```
import type { Slot } from "../types";

// Colour map of all slots: green = free, red = occupied
export default function SlotMap({ slots }: { slots: Slot[] }) {
  return (
```

```
    <div className="bg-white rounded-xl shadow p-4">
      <h2 className="font-semibold mb-3">Slot Map</h2>
      {slots.length === 0 ? (
        <p className="text-sm text-slate-500">
          No slots yet. Open the "Slot Setup" page and draw slots on the camera picture.
        </p>
      ) : (
        <div className="grid grid-cols-3 sm:grid-cols-4 gap-2">
          {slots.map((s) => (
            <div
              key={s.id}
              title={`Updated: ${s.updated_at}`}
              className={`rounded-lg p-3 text-center text-white font-semibold ${
                s.status === "free" ? "bg-green-500" : "bg-red-500"
              }`}
            >
              <div>{s.slot_code}</div>
              <div className="text-xs font-normal uppercase">{s.status}</div>
            </div>
          ))}
        </div>
      )}
    </div>
  );
}
```

11.9 src/components/SourcePanel.tsx

Text box + button to send a camera source (`0` or an RTSP URL) to `/api/source`, and a file input that uploads a photo/video to `/api/source/upload`.

```
import { useState } from "react";
import { setSource, uploadSource } from "../api";

// Lets the operator choose what the AI watches: webcam, file, CCTV URL or an upload
export default function SourcePanel({ current }: { current: string }) {
  const [text, setText] = useState("");
  const [message, setMessage] = useState("");

  const apply = async () => {
    if (!text.trim()) return;
    try {
      await setSource(text.trim());
      setMessage("Camera source changed.");
      setText("");
    } catch (e) {
      setMessage((e as Error).message);
    }
  };

  const upload = async (file?: File) => {
    if (!file) return;
    setMessage("Uploading and searching for parking spaces...");
    try {
      const r = await uploadSource(file);
      setMessage(
        r.error
          ? `"${file.name}" uploaded. ${r.error}`
          : `"${file.name}" uploaded - found ${r.slots} parking spaces (${r.free} free) and ${r.cars}
vehicles.`
      );
    } catch (e) {
      setMessage((e as Error).message);
```

```
    }
  };

  return (
    <div className="bg-white rounded-xl shadow p-4 space-y-3">
      <h2 className="font-semibold">Camera Source</h2>
      <p className="text-sm">
        Current: <span className="font-mono bg-slate-100 px-1 rounded">{current || "-"}</span>
      </p>

      <div className="flex gap-2">
        <input
          value={text}
          onChange={(e) => setText(e.target.value)}
          placeholder='0 = webcam, or rtsp://... camera URL'
          className="flex-1 border rounded-lg px-3 py-2 text-sm"
        />
        <button onClick={apply} className="bg-blue-600 text-white px-4 rounded-lg text-sm">
          Use
        </button>
      </div>

      <p className="text-sm text-slate-500">Or upload a parking-lot photo / video:</p>
      <label className="block w-full text-center cursor-pointer bg-green-600 hover:bg-green-700 text-
white font-semibold py-3 rounded-lg">
        Upload Photo / Video
        <input
          type="file"
          accept="image/*,video/*"
          onChange={(e) => {
            upload(e.target.files?.[0]);
            e.target.value = ""; // allow choosing the same file again
          }}
          className="hidden"
        />
      </label>

      {message && <p className="text-sm text-blue-700">{message}</p>}
    </div>
  );
}
```

11.10 src/components/LogsTable.tsx

Table of the latest `detection_logs` rows. "Outside slots" is highlighted in amber when it is above 0.

```
import type { Log } from "../types";

// History of detections saved by the AI in the detection_logs table
export default function LogsTable({ logs }: { logs: Log[] }) {
  return (
    <div className="bg-white rounded-xl shadow p-4 overflow-x-auto">
      <h2 className="font-semibold mb-3">Detection History</h2>
      <table className="w-full text-sm">
        <thead>
          <tr className="text-left text-slate-500 border-b">
            <th className="py-2">Time</th>
            <th>Total</th>
            <th>Free</th>
            <th>Occupied</th>
            <th>Vehicles</th>
            <th>Outside slots</th>
          </tr>
```

```
        </thead>
        <tbody>
          {logs.map((l) => (
            <tr key={l.id} className="border-b last:border-0">
              <td className="py-2">{l.created_at}</td>
              <td>{l.total_slots}</td>
              <td className="text-green-600 font-medium">{l.free_slots}</td>
              <td className="text-red-600 font-medium">{l.occupied_slots}</td>
              <td>{l.cars_detected}</td>
              <td className={l.cars_outside_slots > 0 ? "text-amber-600 font-medium" : ""}>
                {l.cars_outside_slots}
              </td>
            </tr>
          ))}
          {logs.length === 0 && (
            <tr>
              <td colSpan={6} className="py-3 text-slate-500">No history yet.</td>
            </tr>
          )}
        </tbody>
      </table>
    </div>
  );
}
```

11.11 src/components/SlotSetup.tsx - drawing the slots

This is how the "predefined slot coordinates" are created.

- It loads a **raw** snapshot (`annotated=false`) and, in `onLoad`, stores the picture's real size (`naturalWidth/naturalHeight`), e.g. 1280×720.
- An **SVG** layer is placed exactly on top of the image with `viewBox="0 0 1280 720"`, so the SVG uses the same pixel coordinates as the camera picture, no matter how big the image is shown on screen.
- `toImagePoint()` converts a mouse position on screen into picture pixels: `(mouseX - left) / shownWidth × realWidth`. This is important - YOLO works on the real picture, so slots must be stored in real pixels.
- **Rectangle mode:** `onMouseDown` stores the start point, `onMouseMove` updates the yellow dashed draft, `onMouseUp` sends the slot. Leaving the picture cancels it.
- **4-corner mode:** each `onClick` adds a corner (yellow dots and a line to the mouse are drawn); the 4th click sends the slot with `points`.
- `nextCode()` turns A1 into A2 with a regular expression; `loadSlots()` also skips any code that already exists, so duplicate names cannot happen.
- Saved slots are drawn as blue polygons with their code; line width and text size are relative to the picture width so they look the same on any photo.
- The picture is limited to 75% of the window height (`max-h-[75vh]`) so the whole lot is visible without scrolling.

```
import { useEffect, useRef, useState } from "react";
import type { MouseEvent } from "react";
import { addSlot, autoSlots, deleteAllSlots, deleteSlot, frameUrl, getSlots } from "../api";
import type { Slot } from "../types";

interface Point {
  x: number;
  y: number;
}
```

```
type Shape = "rect" | "corners";

// "A1" -> "A2", "B9" -> "B10"
const nextCode = (c: string) => {
  const m = c.match(/^(.*?)(\d+)$/);
  return m ? m[1] + (parseInt(m[2]) + 1) : c + "1";
};

// Operator marks every parking space on a camera snapshot, either by dragging a
// rectangle or by clicking its 4 corners (for tilted / diagonal spaces).
// The shapes are saved in MySQL and the AI compares cars against them.
export default function SlotSetup() {
  const [snapshot, setSnapshot] = useState(frameUrl(false));
  const [size, setSize] = useState({ w: 0, h: 0 }); // real picture size in pixels
  const [slots, setSlots] = useState<Slot[]>([]);
  const [code, setCode] = useState("A1");
  const [shape, setShape] = useState<Shape>("rect");
  const [start, setStart] = useState<Point | null>(null); // rectangle: drag start
  const [current, setCurrent] = useState<Point | null>(null); // mouse position
  const [corners, setCorners] = useState<Point[]>([]); // 4-corner mode: clicks so far
  const [message, setMessage] = useState("");
  const svgRef = useRef<SVGSVGElement>(null);

  const loadSlots = () =>
    getSlots()
      .then((list) => {
        setSlots(list);
        // skip codes that are already used, so the next slot never gets a duplicate name
        const used = new Set(list.map((s) => s.slot_code));
        setCode((c) => {
          while (used.has(c)) c = nextCode(c);
          return c;
        });
      })
      .catch(() => setMessage("Backend is offline"));

  useEffect(() => {
    loadSlots();
  }, []);

  // convert a mouse position on screen into picture pixel coordinates
  const toImagePoint = (e: MouseEvent): Point => {
    const box = svgRef.current!.getBoundingClientRect();
    return {
      x: Math.round(((e.clientX - box.left) / box.width) * size.w),
      y: Math.round(((e.clientY - box.top) / box.height) * size.h),
    };
  };

  const save = async (slot: Omit<Slot, "id" | "status" | "updated_at">) => {
    try {
      await addSlot(slot);
      setMessage(`Slot ${slot.slot_code} saved.`);
      setCode(nextCode(slot.slot_code));
      loadSlots();
    } catch (e) {
      setMessage((e as Error).message);
    }
  };

  // ----- rectangle mode: press, drag, release -----
  const finishRect = () => {
    if (shape !== "rect" || !start || !current) return;
    setStart(null);
    save({ slot_code: code, x1: start.x, y1: start.y, x2: current.x, y2: current.y, points: null });
```

```
};

// ----- 4-corner mode: click 4 times -----
const addCorner = (e: MouseEvent) => {
  if (shape !== "corners") return;
  const list = [...corners, toImagePoint(e)];
  if (list.length < 4) {
    setCorners(list);
    return;
  }
  setCorners([]);
  const xs = list.map((p) => p.x);
  const ys = list.map((p) => p.y);
  save({
    slot_code: code,
    x1: Math.min(...xs),
    y1: Math.min(...ys),
    x2: Math.max(...xs),
    y2: Math.max(...ys),
    points: list.map((p) => [p.x, p.y]),
  });
};

const changeShape = (s: Shape) => {
  setShape(s);
  setStart(null);
  setCorners([]);
};

const removeSlot = async (id: number) => {
  await deleteSlot(id);
  loadSlots();
};

const autoDetect = async () => {
  if (slots.length > 0 && !confirm("This replaces ALL saved slots. Continue?")) return;
  setMessage("Searching the picture for parking spaces...");
  try {
    const r = await autoSlots();
    setMessage(`Found ${r.slots} parking spaces (${r.free} free) and ${r.cars} vehicles.`);
    setSnapshot(frameUrl(false));
    loadSlots();
  } catch (e) {
    setMessage((e as Error).message);
  }
};

const removeAll = async () => {
  if (!confirm("Delete ALL slots?")) return;
  await deleteAllSlots();
  setCode("A1");
  loadSlots();
};

// line width / text size relative to the picture, so they look the same on any photo
const stroke = Math.max(2, size.w / 400);
const fontSize = Math.max(14, size.w / 45);
const toSvg = (pts: number[][]) => pts.map((p) => p.join(",")).join(" ");

const shapeButton = (s: Shape, label: string) => (
  <button
    onClick={() => changeShape(s)}
    className={`px-3 py-1 rounded-lg text-sm ${
      shape === s ? "bg-blue-600 text-white" : "bg-slate-200"
    }`}
  >
```

```
      {label}
    </button>
  );

  return (
    <div className="grid lg:grid-cols-3 gap-6">
      <div className="lg:col-span-2 bg-white rounded-xl shadow p-4">
        <div className="flex flex-wrap items-center justify-between gap-2 mb-3">
          <div className="flex gap-2">
            {shapeButton("rect", "Rectangle (drag)")}
            {shapeButton("corners", "4 corners (click)")}
          </div>
          <button
            onClick={() => setSnapshot(frameUrl(false))}
            className="bg-slate-200 px-3 py-1 rounded-lg text-sm"
          >
            New snapshot
          </button>
        </div>
        <p className="text-sm text-slate-500 mb-3">
          {shape === "rect" ? (
            <>
              Press on one corner of a <b>whole</b> parking space, drag to the opposite corner and
              release. Best for straight spaces.
            </>
          ) : (
            <>
              Click the <b>4 corners</b> of one parking space (on its painted lines). After the 4th
              click the slot is saved. Best for diagonal / tilted spaces.
              {corners.length > 0 && <b> Corners clicked: {corners.length}/4</b>}
            </>
          )}
        </p>

        {/* whole picture fits on screen (max 75% of window height) so no scrolling while drawing */}
        <div className="relative select-none w-fit mx-auto">
          <img
            src={snapshot}
            alt="Camera snapshot"
            className="block max-w-full max-h-[75vh] rounded-lg bg-slate-200 min-h-40"
            onLoad={(e) =>
              setSize({ w: e.currentTarget.naturalWidth, h: e.currentTarget.naturalHeight })
            }
            onError={() => setMessage("No camera frame yet - start the backend and wait a few
seconds.")}
          />
          {size.w > 0 && (
            <svg
              ref={svgRef}
              viewBox={`0 0 ${size.w} ${size.h}`}
              preserveAspectRatio="none"
              className="absolute inset-0 w-full h-full cursor-crosshair"
              onMouseDown={(e) => {
                if (shape !== "rect") return;
                const p = toImagePoint(e);
                setStart(p);
                setCurrent(p);
              }}
              onMouseMove={(e) => setCurrent(toImagePoint(e))}
              onMouseUp={finishRect}
              onClick={addCorner}
              onMouseLeave={() => setStart(null)} // left the picture -> cancel the rectangle
            >
              {slots.map((s) => (
                <g key={s.id}>
                  <polygon
```

```
                  points={toSvg(s.points ?? [[s.x1, s.y1], [s.x2, s.y1], [s.x2, s.y2], [s.x1, s.y2]])}
                  fill="rgba(37,99,235,0.25)"
                  stroke="#2563eb"
                  strokeWidth={stroke}
                />
                <text
                  x={s.x1 + fontSize / 3}
                  y={s.y1 + fontSize}
                  fill="#fff"
                  stroke="#1e3a8a"
                  strokeWidth={stroke / 2}
                  fontSize={fontSize}
                  fontWeight="bold"
                >
                  {s.slot_code}
                </text>
              </g>
            ))}

            {/* rectangle being dragged */}
            {shape === "rect" && start && current && (
              <rect
                x={Math.min(start.x, current.x)}
                y={Math.min(start.y, current.y)}
                width={Math.abs(current.x - start.x)}
                height={Math.abs(current.y - start.y)}
                fill="rgba(250,204,21,0.3)"
                stroke="#facc15"
                strokeWidth={stroke}
                strokeDasharray={`${stroke * 3} ${stroke * 2}`}
              />
            )}

            {/* corners clicked so far + line to the mouse */}
            {shape === "corners" && corners.length > 0 && (
              <>
                <polyline
                  points={toSvg([...corners, ...(current ? [current] : [])].map((p) => [p.x, p.y]))}
                  fill="rgba(250,204,21,0.3)"
                  stroke="#facc15"
                  strokeWidth={stroke}
                />
                {corners.map((p, i) => (
                  <circle key={i} cx={p.x} cy={p.y} r={stroke * 2.5} fill="#facc15" />
                ))}
              </>
            )}
          </svg>
        )}
      </div>
    </div>

    <div className="bg-white rounded-xl shadow p-4 space-y-4">
      <button
        onClick={autoDetect}
        className="w-full bg-green-600 hover:bg-green-700 text-white font-semibold py-3 rounded-lg"
      >
        Auto-detect parking spaces
      </button>
      <p className="text-xs text-slate-500">
        The AI finds spaces from the white painted lines and around every parked car it sees.
        Empty spaces without clear lines can't be found - add them by hand afterwards.
      </p>
      <label className="block text-sm font-medium">
        Code for the next slot
        <input
```

```
            value={code}
            onChange={(e) => setCode(e.target.value)}
            className="block w-full border rounded-lg px-3 py-2 mt-1"
          />
        </label>
        {message && <p className="text-sm text-blue-700">{message}</p>}

        <div>
          <div className="flex items-center justify-between mb-2">
            <h3 className="font-semibold">Saved slots ({slots.length})</h3>
            {slots.length > 0 && (
              <button onClick={removeAll} className="text-xs text-red-600">
                Delete all
              </button>
            )}
          </div>
          <ul className="divide-y text-sm max-h-96 overflow-y-auto">
            {slots.map((s) => (
              <li key={s.id} className="py-2 flex items-center justify-between">
                <span>
                  <b>{s.slot_code}</b>{" "}
                  <span className="text-slate-500">{s.points ? "4 corners" : "rectangle"}</span>
                </span>
                <button onClick={() => removeSlot(s.id)} className="text-red-600 text-xs">
                  Delete
                </button>
              </li>
            ))}
          </ul>
        </div>
      </div>
    </div>
  );
}
```

11.12 index.html in the project root

Redirects `http://localhost/smart-parking/` to the built site in `app/`.

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="refresh" content="0; url=app/" />
    <title>Smart Parking</title>
  </head>
  <body>
    <a href="app/">Open Smart Parking</a>
  </body>
</html>
```

12. Changing the code and rebuilding

Frontend

The website in `app/` is generated. After editing anything in `frontend/src`:

```
cd C:\xampp\htdocs\smart-parking\frontend
npm install        (only the first time)
npm run build      (rebuilds app\  -> refresh the browser)
```

While developing you can instead run `npm run dev` and open `http://localhost:5173`; changes appear instantly.

Backend

Start Uvicorn with `--reload` (`uvicorn main:app --reload --port 8000`) and it restarts automatically when you save a .py file.

13. Testing checklist

#	Test	Expected result
1	Open `http://localhost:8000/`	`{"message": "Smart Parking API is running..."}`
2	Open phpMyAdmin	Database `smart_parking` with 2 tables exists
3	Open the website with backend stopped	Red "Cannot reach the backend" message
4	Upload a parking-lot photo	Live picture shows orange boxes around cars; "Vehicles Seen" > 0
5	Draw 3 slots: 2 on cars, 1 on an empty space	Slots saved; dashboard shows Total 3, Occupied 2, Free 1
6	Upload a video where a car leaves	That slot turns from red to green within ~2 s; `updated_at` changes in MySQL
7	Draw a slot with an existing code	Error "Slot code ... already exists"
8	Set source to `99` (no such camera)	Yellow AI error message; set `0` again and it recovers
9	Wait 1 minute	About 6 new rows in Detection History

14. Troubleshooting

Problem	Cause / fix
`'python' is not recognized`	Python not installed or not on PATH. Reinstall Python 3.12 and tick "Add python.exe to PATH".
`Can't connect to MySQL server on 'localhost'`	Start MySQL in the XAMPP Control Panel. If you set a root password, put it in `config.py` → `DB_PASSWORD`.
Website shows "Cannot reach the backend"	Uvicorn is not running or is on another port. Run `uvicorn main:app --port 8000` inside `backend` with the venv activated.
`Cannot read from camera source: 0`	No webcam, or another app (Zoom/Teams) is using it. Upload a photo/video instead.
Cars are not detected / few boxes	Wrong model for the camera angle. Top-down photo → `YOLO_MODEL = "yolov8n-obb.pt"`, `IMAGE_SIZE = 1024`. Angled CCTV → `YOLO_MODEL = "yolov8n.pt"`, `IMAGE_SIZE = 640`. Restart the backend after changing. Lower `CONFIDENCE` to 0.2 for more boxes, or use a bigger model (`yolov8s-obb.pt` / `yolov8s.pt`).
Slot shows occupied by the neighbouring car	Raise `OCCUPIED_OVERLAP` (e.g. 0.45) or draw slots slightly smaller.
Website is blank / 404	Apache not started, or `app\` missing → run `npm run build` in `frontend`.
Slots are in the wrong place after changing camera	Slot coordinates belong to one camera view. Delete all and draw again.
Port 80 is busy (Apache won't start)	Skype/IIS uses it. In XAMPP change Apache's port to 8080 and open `http://localhost:8080/smart-parking/`.

15. What to build next

- **Reservations**: a `reservations` table (user, slot, start, end, status) and a lifecycle (reserved → checked-in → completed / cancelled), waitlist and automatic cancellation.
- **Authentication**: Supabase Auth (or a `users` table) for drivers and operators.
- **Overstay anomaly detection** (Feature 2): use `slots.updated_at` and the reservation end time; Scikit-learn `IsolationForest` on parking durations to flag abandoned vehicles.
- **Heatmap of multiple lots**: add a `parking_lots` table and a `lot_id` column in `slots`, then show lots on a map coloured by free-slot percentage.
- **Custom-trained model**: train YOLO on a parking dataset (PKLot, CNRPark-EXT) with the classes "space-empty" / "space-occupied". This detects free spaces directly, even without painted lines, and recognises cars that the pre-trained models miss.
- **Accuracy**: require a slot to have the same status for 2-3 frames in a row before changing it (removes flicker); use polygons instead of rectangles for angled cameras.

- **CI/CD**: GitHub Actions running `ruff` + `pytest` (backend) and `npm run build` (frontend); deploy to Vercel / Render.